1(a)	Answers include: 1 They are tried and tested // Free from errors 2 They are readily available // Speed up development time 3 They will be updated automatically when improvements are made // No maintenance of routine is needed (as automatically updated) 4 The algorithm will be compliant with the international standard One mark per point **Max 3**	**3**

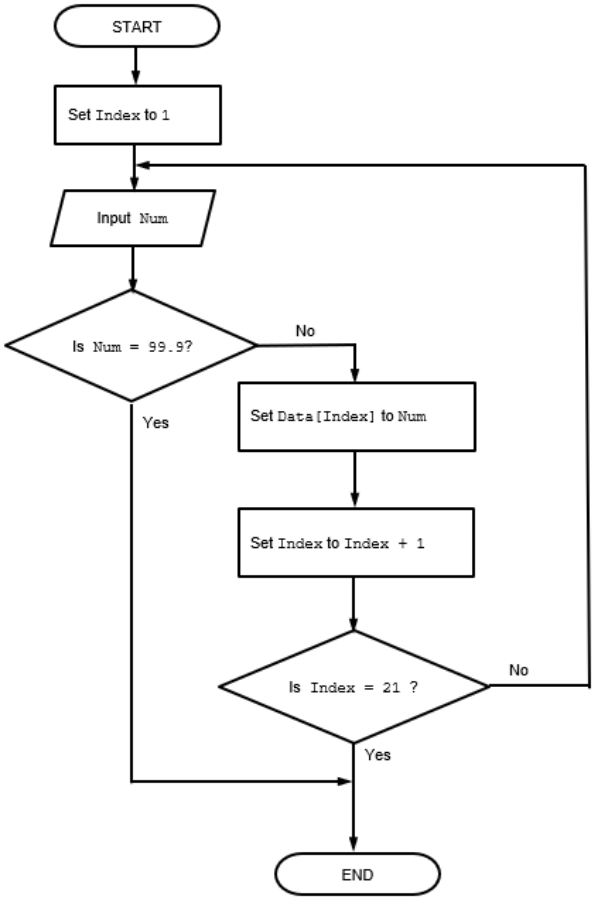
1(b)(i)	1 When a part of the algorithm performs a specific task 2 Part is repeated // performed in several places 3 Testing / debugging / maintenance is easier One mark for each point **Max 2**	**2**

Examiner report missing
9618_w25_qp_23 Question 1

2(a)	One mark per point: 1 Open the file in append mode (and subsequently close) **before loop** 2 Initialise a variable to 1 / 0 **before loop** 3 Use the variable as an index to the array to access an element **in a loop** 4 Test if element value is blank / empty **in a loop** 5 If not blank then write the value to the file **in a loop** 6 Increment the variable **in a loop** 7 Repeat from MP3 until variable value is 66 / 65 / all elements (in the array) have been checked **Max 6**	**6**

2(b)	One mark per point: Construct: Selection Use: To test whether the (array) element is blank Alternative: Construct: Sequence Use: To specify the order in which the steps of the algorithm have to be followed so that the task is completed correctly	**2**

Examiner report missing
9618_w25_qp_23 Question 2

2	 Mark as follows: **MP1** Initialisation of the array index **MP2** Input number **in a loop** **MP3** Test for termination value (99.9) **and** correct ending **MP4** Test for array full / maximum 20 iterations **and** correct ending **MP5** Assign number to `Data` array element **and** increment index	**5**

Examiner report missing
9618_w25_qp_22 Question 2

3(a)	Mark as follows:		**6**

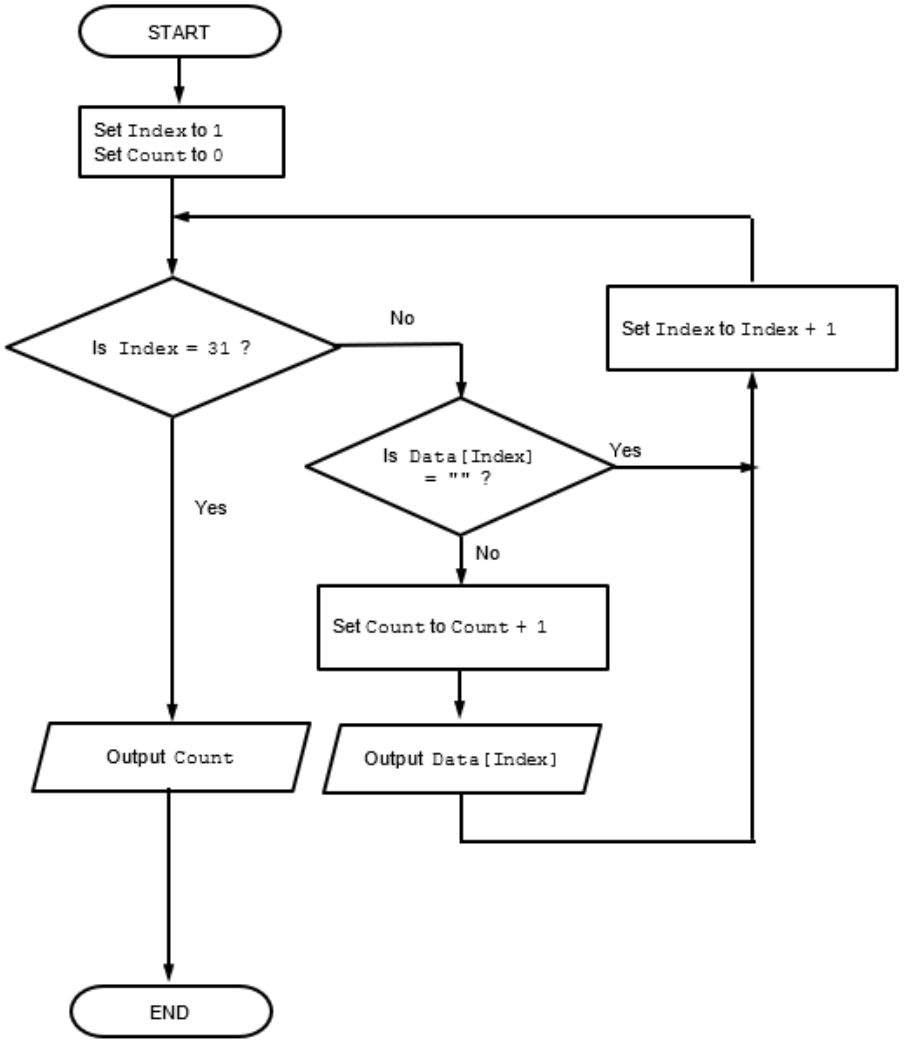
MP1	1		open the file **OldFile.txt** in **read (mode)**
MP2	2		open the file **NewFile.txt** in **write (mode)**
	3		read a line from **OldFile.txt** and store in **`LineX`**
MP3	4		read a line from **OldFile.txt** and store in **`LineY`**
	5		read a line from **OldFile.txt** and store in **`LineZ`**
MP4	6		set `NewString` to **`LineX & '\' & LineY & '\' & LineZ`**
MP5	7		write `NewString` to **NewFile.txt**
MP6	8		repeat from step **3** until **end of OldFile.txt**
	9		close both files

Examiner report missing
9618_w25_qp_22 Question 3

4(a)	Example solution:
	
	`DECLARE CheckTotal : ARRAY [0:35] OF BOOLEAN`
	`DECLARE Index : INTEGER`
	
	`FOR Index ← 0 TO 35`
	`  CheckTotal[Index] ← FALSE`
	`NEXT Index`
	
	Mark as follows:
	
	MP1 Declaration of `CheckTotal` array
	MP2 Declaration of a loop counter
	MP3 Loop – with correct syntax and number of iterations
	MP4 Assignment of array element to `FALSE`
		4

4(c)	**MP1** Check that the parameter value is in the range 0 to 35 **MP2** ... and if not, return `FALSE` **MP3** Use the **parameter** value as **the index to array** `CheckTotal` **MP4** Return the value from given index position	**4**

Examiner report missing
9618_w25_qp_22 Question 4

2	Example solution  Mark as follows: 1 Initialisation of variables used as **array index** and **count** 2 Loop through exactly 30 elements in the array 3 Test for empty string in current array element // Test for non-empty string 4 Increment count if appropriate **in a loop** 5 Both required outputs under correct conditions with correct outputs	**5**

Examiner report missing
9618_w25_qp_21 Question 2

5(a)(i)	It is a value that is **invalid as an** (array) **index** // not in the **range** of valid (index) values	**1**

5(a)(iii)	The value of `FoundAt` is only updated to the (current) index if it currently stores the initial value / -1 // `FoundAt` is not updated when it currently stores any other value than -1	**1**

Examiner report missing
9618_w25_qp_21 Question 5

6(a)	Number:	Any number where the first two digits sum to 5 / 15 **and** the last three digits are 623	**2**
	Explanation:	The remainder is the same / 3 // The sum of the (first four) digits is the same / 13	

Examiner report missing
9618_w25_qp_21 Question 6

1(a)(i)	**MP1** **MP2** **MP3** **MP4** **MP5** **Max 2**	When a **task/module is repeated** / reused / called // performed in several places A **specific task** can be identified /coded as a module / subroutine / procedure / function. Reduces complexity of **program / code** // **Program / code** is simplified Module / subroutine / procedure / function **already available**. Testing / debugging / maintenance is easier	**2**

Examiner report missing
9618_s25_qp_23 Question 1

4(a)	**MP1** **Open** the file (`Sales.txt`) in **read mode and** subsequently close **MP2** **Read <u>a line</u>** (from the text file) **MP3** Convert the line read (string) to a number **MP4** If the number is greater than 500 **and** then output week number **MP5** Increment week number (must have been Initialised **…**) **MP6** Repeat from step 2 for 52 times // Repeat from step 2 until end of file **Max 5**	**5**

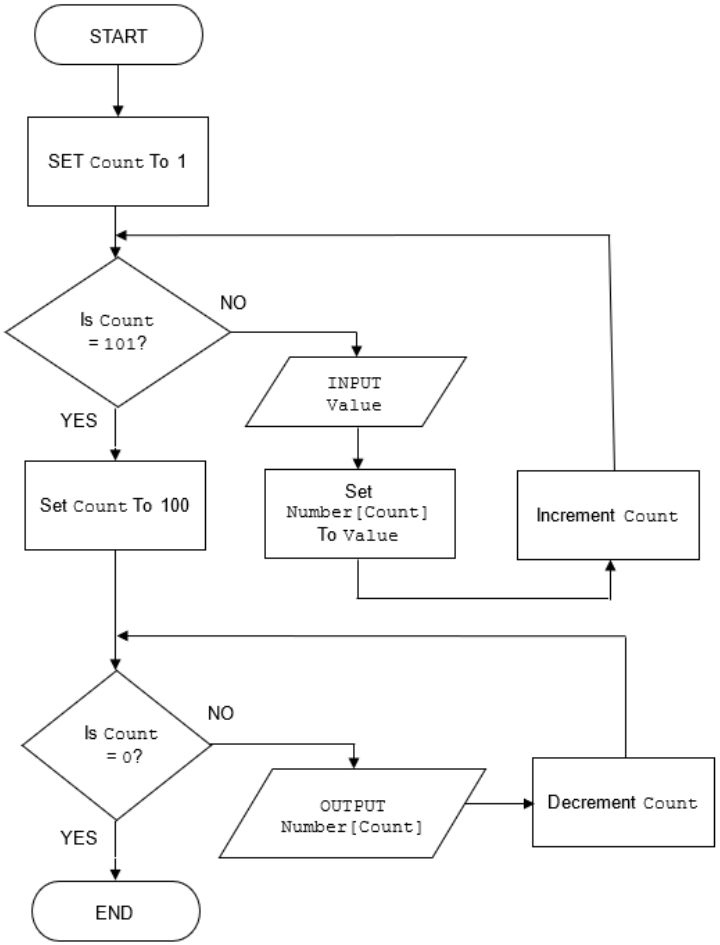
Examiner report missing
9618_s25_qp_23 Question 4

2(a)	Mark as follows: 1. To increase the level of detail of the algorithm // To break the problem / task into **smaller steps** … 2. … until steps can be directly translated into lines of code // from which it can be programmed	**2**

Examiner report missing
9618_s25_qp_22 Question 2

3	Example solution
	
	`DECLARE FirstNumber : INTEGER`
	`DECLARE SecondNumber : INTEGER`
	`DECLARE ThirdNumber : INTEGER`
	
	`FirstNumber ← INT(RAND(21)) - 10`
	
	`OUTPUT FirstNumber`
	
	`REPEAT`
	`  SecondNumber ← INT(RAND(21)) - 10`
	`UNTIL FirstNumber <> SecondNumber`
	
	`OUTPUT SecondNumber`
	
	`IF FirstNumber < 0 AND SecondNumber < 0 THEN`
	`  ThirdNumber ← INT(RAND(6)) + 30`
	`  OUTPUT ThirdNumber`
	`ENDIF`
	
	Mark points:
	
	1. Declare all variables used
	2. Use `RAND()` function with any **integer** parameter
	3. Use `INT()` function with any **numeric** parameter
	4. Conditional loop until **two different** random numbers are generated
	5. Use `INT()` function using random number generated between –10 and 10 inclusive **in a loop**
	6. Output **two different** random integers / numbers
	7. Check if both random integers / numbers are negative
	8. then output a (third) random integer / number between 30 and 35 inclusive
		8

Examiner report missing
9618_s25_qp_22 Question 3

4	 One mark for each functional group as listed: 1. Initialise Index used for `Number` to either 1 or 0 before first loop 2. Loop 100 times 3. Input value 4. Increment array index **and** store value in `Number` array at element referenced by array `index` **in a loop** 5. Output loop starts at last element in `Number` array 6. Output `Number` array element referenced by `Index` **in a loop** 7. Output all elements of `Number` array **once** in reverse order **Max 6**	**6**

Examiner report missing
9618_s25_qp_22 Question 4

1(a)

Example value	Explanation	Variable name	Data type
"Fruit"	a category of stock that is sold in the shop	**Category**	**STRING**
20/02/2025	when an item was sold	**DateSold**	**DATE**
12.67	the cost of an item	**(Item)Cost/Price**	**REAL**
TRUE	to indicate if an item is in stock	**(Item)InStock**	**BOOLEAN**

Mark as follows:
1 mark per row for each variable name **and** data type

4

1(b)

Pseudocode extract	Assignment	Selection	Iteration
`Result ← CalculateTotal()`	✓		
`WHILE IsClosed`			✓

`REPEAT`
`  INPUT Value`
`UNTIL Sales[4] > Value`
	✓		✓

`IF Sales[Current] <= 150 THEN`
`  Discount ← TRUE`
`ENDIF`
	✓	✓	

`CASE OF Option`		✓	

Mark as follows:
One mark for each correct row

5

1(c)	Decomposition involves: **MP1** Breaking down a problem into **sub problems** **MP2** In order to explain / understand **the process/task** of how a (stock control) can be managed in more detail **MP3** Leading to the concept of the program designed as a series of **modules** / procedures / functions // or **by example** of different **stock control** modules **MP4** Makes the **program** easier to **test / debug** **Max 3** **Max 2** if no mention of stock control	**3**

Examiner report missing
9618_s25_qp_21 Question 1

3	Example solution:
	
	`DECLARE GeneratedValue : INTEGER`
	`DECLARE Guess : INTEGER`
	
	`GeneratedValue ← INT(RAND(100)) + 1`
	
	`REPEAT`
	`  OUTPUT "Enter a guess between 1 and 100: "`
	`  INPUT Guess`
	
	`  IF Guess > GeneratedValue THEN`
	`  OUTPUT "Guess was too high"`
	`  ENDIF`
	`  IF Guess < GeneratedValue THEN`
	`  OUTPUT "Guess was too low"`
	`  ENDIF`
	`UNTIL Guess = GeneratedValue`
	
	`OUTPUT "Number guessed correctly"`
	
	MP1 Declare all variables used
	
	MP2 Uses `RAND()` function
	MP3 Uses `INT()` function
	MP4 Correct syntax for random number between 1 and 100
	
	MP5 Conditional loop until correct number guessed
	MP6 Prompt and input a guess **in a loop**
	
	MP7 Check if guess is too high **in a loop**
	MP8 Check if guess is too low **in a loop**
	
	MP9 Output appropriate message (x2) when the number is **not** guessed correctly (**in** the loop) **and** output correct guess message
	
	Max 8
		8

Examiner report missing
9618_s25_qp_21 Question 3

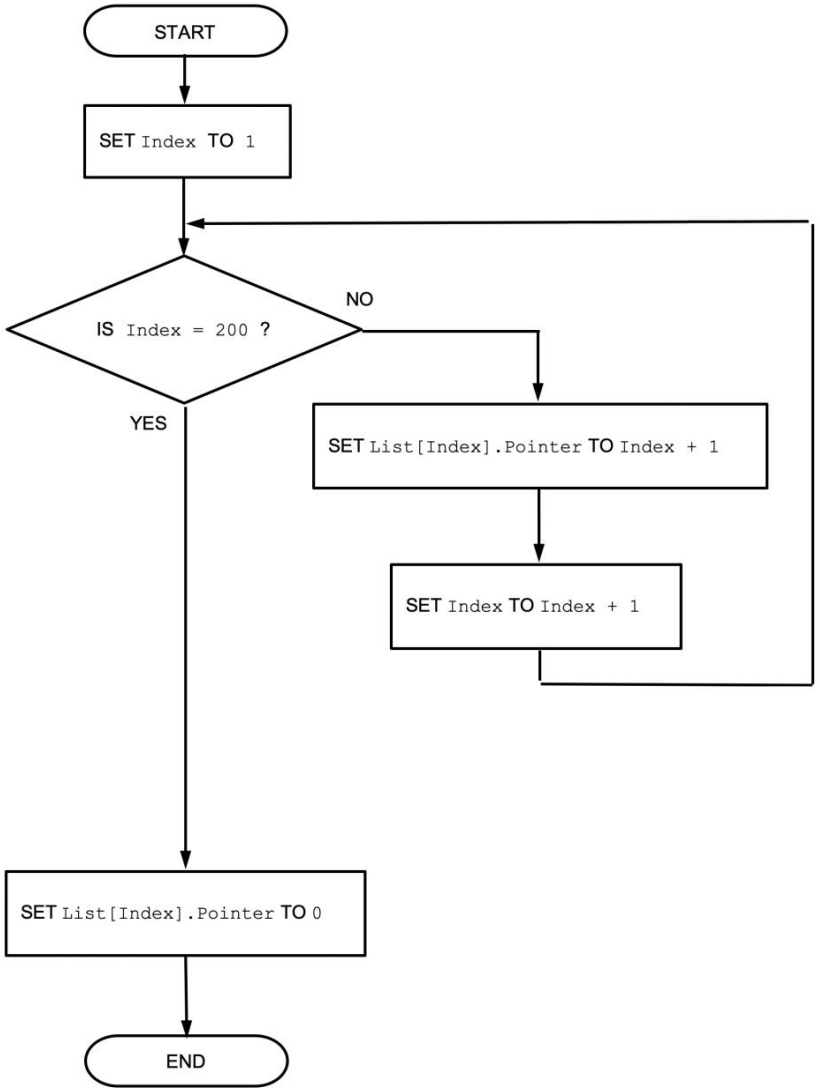
Examiner report missing
9618_w24_qp_23 Question 1

2(a)	`DECLARE Count, Total, NextNumber : INTEGER`
	
	`Total ← 0`
	`FOR Count ← 1 TO 100`
	`  OUTPUT "Input an integer value"`
	`  INPUT NextNumber`
	`  IF NextNumber > 0 THEN`
	`  Total ← Total + NextNumber`
	`  ENDIF`
	`NEXT Count`
	
	`OUTPUT Total`
	
	Mark as follows:
	MP1 Declarations of all variables used
	MP2 Loop for 100 iterations
	MP3 Prompt **and** input a value **in a loop and**
	MP4 Test for value > 0 // >=1 **in a loop**
	MP5 Sum the Total **in a loop**
	MP6 Output of Total after the **loop**
	
	Max 5 marks
		5

2(b)	**MP1** Construct: Iteration / Repetition **MP2** Use: To loop through all 100 inputs // To loop 100 times ALTERNATIVE: **MP1** Construct: Selection **MP2** Use: To test whether the value input is positive	**2**

Examiner report missing
9618_w24_qp_23 Question 2

3(a)(i)	0 is not a valid <u>array / `List` index</u> value // No 0th element in the array	**1**

3(b)	 Mark as follows: **MP1** Use of variable as array index **and** initialisation to 1 / to first element of array **MP2** Loop for 199 / 200 iterations **MP3** Assign Index+1 to each pointer field in a loop **MP4** Assign 0 to 200th pointer field	**4**

3(c)	One mark per step: **MP1** Assign `HeadPointer` to `ThisPointer` / Current Pointer // Identify first node using headpointer **MP2** Loop the following until `ThisPointer` value is 0 (zero) / null pointer reached **MP3** … Output data (field) from array element at index `ThisPointer` // Output data (field) of the current node / array element **MP4** … Assign pointer field from current array element / index / to `ThisPointer` / Current Pointer // Assign pointer field from current node to `ThisPointer` / Current Pointer	**4**

Examiner report missing
9618_w24_qp_23 Question 3

7(a)	One mark per answer: **MP1** Analysis **MP2** **Named** document, examples include: • A requirement specification • Definition of System **Objectives** • List of problems with existing system • Survey results • Feasibility study • Interview notes **Max 2 marks**	**2**

7(b)	One mark for name **and** use Mark as follows: Examples include: Sub-module: `ScanCard()/ GetID()` Use: Read the barcode from the loyalty card / Get the customer ID from the barcode Sub-module: `GetPoints()` Use: Get the number of points the customer has Sub-module: `ExchangePoints()` / `UpdateCard()` Use: Reduce the number of loyalty points Sub-module: `GetDiscount()` / `CalculateDiscount()` Use: Calculate the discount Sub-module: `CalculatePoints()` Use: Calculate points (following a purchase) Sub-module: `UpdatePoints()` Use: Update total points a customer has (following a purchase) Sub-module: `ShowDiscount()` Use: Display the discount **Max 4 marks**	**4**

Examiner report missing
9618_w24_qp_23 Question 7

6(a)	Example solution:
	
	`PROCEDURE Special()`
	`  DECLARE Index : INTEGER`
	`  DECLARE Filling1, Filling2 : STRING`
	
	`  REPEAT`
	`  Index ← INT(RAND(35)) + 1`
	`  UNTIL Filling[Index] <> ""`
	
	`  Filling1 ← Filling[Index]`
	
	`  REPEAT`
	`  Index ← INT(RAND(35)) + 1`
	`  UNTIL Filling[Index] <> "" AND Filling1 <> Filling[Index]`
	
	`  Filling2 ← Filling[Index]`
	
	`  REPEAT`
	`  Index ← INT(RAND(10) + 1)`
	`  UNTIL Bread[Index] <> ""`
	
	`  OUTPUT "The daily special is ", Filling1, " and ", __ Filling2, " on ", Bread[Index], " bread."`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Loop for Filling 1, avoiding unused elements
	MP2 Loop for Filling 2 avoiding unused elements
	MP3 Check Filling 2 is different from Filling 1 – could correctly compare either the indices or the array contents
	
	MP4 Loop for Bread, avoiding unused elements
	
	MP5 Using `RAND(10) / RAND(35)`
	MP6 Completely correct use of `RAND()` – including `INT()` and +1 in all cases
	
	MP7 Correct output - **once only** – following a reasonable attempt at selection of filings and bread
		7

6(b)	Answers include: **MP1** For each filling, create a list of acceptable / incompatible fillings/indexes **MP2** When selecting the second filling, (as well as checking for an unused element) check that the filling / index is / is not on the list ALTERNATIVE: **MP1** Create a list of 'good' combinations **MP2** Randomly select from this list	**2**

Examiner report missing
9618_w24_qp_22 Question 6

7(a)	• Customer ID – to reference the other customer details • Email address – to send the email • Name – to personalise the email • Date of last visit – to select which customers should receive an email • Unique voucher code (or method of code generation) – to include in the email Mark as follows: One mark per item **and** justification **Max 3 marks**	**3**

7(b)	Abstraction	**1**

Examiner report missing
9618_w24_qp_22 Question 7

3(a)	One mark per emboldened part: 1 Open the file **(Result.txt) in <u>read</u> mode** 2 Loop until **EOF(Result.txt)** // **EOF** // **end of file** 3 **<u>Read a</u> / <u>the</u> / <u>next line</u> (from the file)** and store in `ThisLine` 4 Assign `LineY` to `LineX` 5 Assign `LineZ` to `LineY` 6 Assign `ThisLine` to `LineZ` 7 After the loop, **close the file (Result.txt)** 8 Output `LineX`, `LineY`, `LineZ`	**4**

3(b)	Example answer: So that the last three lines of the file are output in the correct **order** A mark for mentioning one of: • (Ensuring) the lines are **output in correct order** • Ordering the last <u>three</u> lines • Ordering the lines so they are in the same order as (they occur) <u>in the file</u> **Max 1 marks**	**1**

Examiner report missing
9618_w24_qp_21 Question 3

4(a)	One mark per point: **Type:** Run-time error **Cause:** A divide by zero operation is attempted // Attempt to convert a non-numeric string to a number	**2**

4(b)	Answers include: **Reason:** • This part of the algorithm performs a specific task // if the check or calculation is changed it is changed only once • A (similar) subroutine is already available // Library routine is already available • The program is simplified / easier to understand / easier to design / code / test / debug / maintain **Max 1 marks for 'Reason'** **Avoided:** • Unnecessary code duplication • Errors caused by differences where several copies of the check and calculation exist **Max 1 marks for 'Avoided'**	**2**

Examiner report missing
9618_w24_qp_21 Question 4

1(a)(i)	Two marks for the benefits: 1. The code can be called when needed 2. Any subsequent change to the algorithm / calculation needs to be made once only // Easier to manage / maintain (the program) 3. Less code / no code duplication 4. The algorithm / code / calculation can be designed / coded / tested **once** **Note: Max 2 marks**	**2**

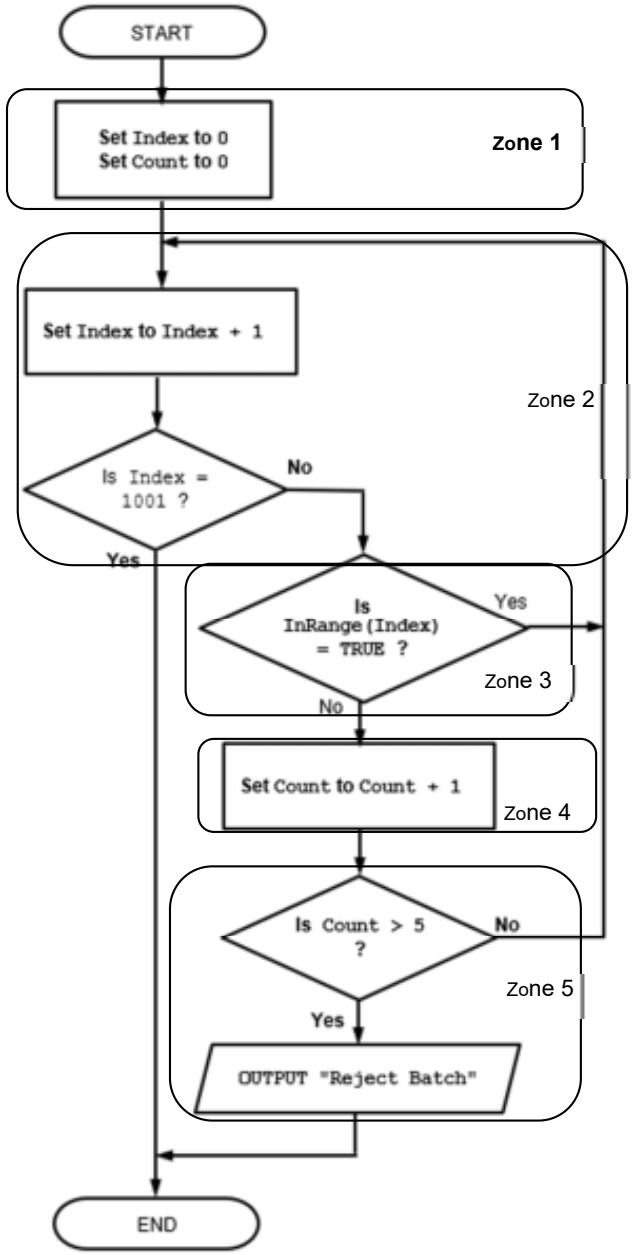
Examiner report missing
9618_s24_qp_23 Question 1

2(a)	One mark per point: 1. Open the file (in read mode and subsequently close) 2. Initialise an index variable (to 1 // 0) 3. Repeat (the next three steps) until the end of file is reached 4. Read a line from the file (into a string variable) 5. Store the line / variable in the array at the index 6. Increment the index **in a loop** **Note: max 5 marks**	**5**

2(b)	One mark per point: Construct: a conditional loop Use: To keep repeating until the **end of the file is reached** ALTERNATIVE: Construct: a selection statement Use: To test / check the value returned by the `EOF()` function	**2**

Examiner report missing
9618_s24_qp_23 Question 2

3(a)(i)	Example solution using AND
	
	`IF Batch[ThisIndex].Weight >= Min AND`
	`Batch[ThisIndex].Weight <= Max THEN`
	
	Alternative solution using OR
	
	`IF Batch[ThisIndex].Weight < Min OR`
	`Batch[ThisIndex].Weight > Max THEN`
	
	Mark as follows:
	1. Reference to `Batch[ThisIndex].Weight`
	2. A valid check for one boundary
	3. A valid check for other boundary with correct logic operator
		3

3(b)	 One mark per zone	**5**

Examiner report missing
9618_s24_qp_23 Question 3

4(a)	Example solution:
	
	`PROCEDURE TwoParts()`
	`  DECLARE NextNum, Average : REAL`
	
	`  TotalA ← 0.0 // 0`
	`  TotalB ← 0.0 // 0`
	
	`  REPEAT`
	`  INPUT NextNum`
	`  TotalA ← TotalA + NextNum`
	`  UNTIL NextNum = 0`
	
	`  REPEAT`
	`  INPUT NextNum`
	`  TotalB ← TotalB + NextNum`
	`  UNTIL NextNum = 0`
	
	`  Average ← (TotalA + TotalB) / 2`
	`  OUTPUT "The average is ", Average`
	`ENDPROCEDURE`
	
	Mark as follows:
	1. Procedure heading and ending
	2. Declare all local variables
	3. Initialise `TotalA` and `TotalB`
	4. First conditional loop until zero entered, summing `TotalA` // Loop until both parts (sequences) have been entered
	5. Second conditional loop until zero entered, summing `TotalB` // Loop summing appropriate Totals
	6. Calculation of average and output with a message // Calculation of the average for the values making up the two totals and both output with a suitable message
		6

Examiner report missing
9618_s24_qp_23 Question 4

1(a)		**4**

Pseudocode example	Selection	Iteration	Input/Output

`FOR Index ← 1 TO 10`
`  Data[Index] ← 0`
`NEXT Index`
		✓	

`WRITEFILE ThisFile, "*****"`			✓
`UNTIL Level > 25`		✓	

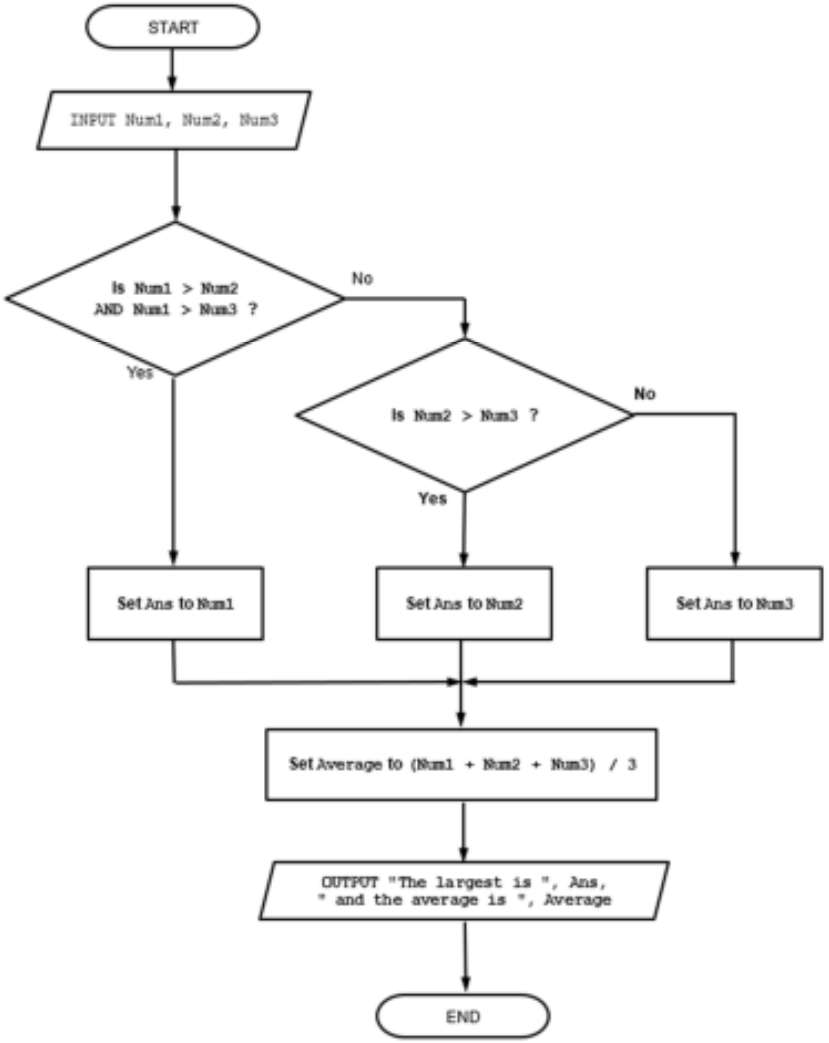
`IF Mark > 74 THEN`
`  READFILE OldFile, Data`
`ENDIF`
	✓		✓

One mark per row.

1(b)	**Expression**
	`MyInt ← INT (3.1415926)`
	`MyChar ← MID ("Elwood", 3, 1)`
	Any of:
	• `MyString ← NUM_TO_STR (INT (27.509))`
	• `MyString ← CHR (INT (27.509))`
	• `MyString ← TO_UPPER( NUM_TO_STR( 27.509))`
	• `MyString ← TO_LOWER( NUM_TO_STR( 27.509))`
	Any of:
	• `MyInt ← STR_TO_NUM ( RIGHT ("ABC123", 3))`
	• `MyInt ← LENGTH ( RIGHT ("ABC123", 3))`
	• `MyInt ← LENGTH ( LEFT ("ABC123", 3))`
	One mark per row
		4

1(c)	1 mark for stating a suitable way of documenting: • Identifier table 1 mark for giving one piece of information that should be recorded. examples include: Explanation of what (each) variable is used for The purpose of (each) variable An example of data values stored // Initialisation value	**2**

Examiner report missing
9618_s24_qp_22 Question 1

2(a)	 Mark points 1 Condition for selecting one of the input numbers as largest value 2 … and assign to `Ans` 3 Condition for selecting largest number for all three number input and assigning to `Ans` 4 Average calculated using `Num1`, `Num2` and `Num3` and stored in a variable. Reject use of DIV 5 Output of `Ans` and average in output symbol / parallelogram	**5**

2(b)	Example solutions:
	
	`Flag ← GetStat()`
	`WHILE Flag <> TRUE`
	`  FOR Port ← 1 TO 3`
	`  CALL Reset(Port)`
	`  NEXT Port`
	`  Flag ← GetStat()`
	`ENDWHILE`
	
	Alternative:
	
	`REPEAT`
	`  Flag ← GetStat()`
	`  IF Flag <> TRUE THEN`
	`  FOR Port ← 1 TO 3`
	`  CALL Reset(Port)`
	`  NEXT Port`
	`  ENDIF`
	`UNTIL Flag = TRUE`
	
	One mark per point:
	1 (Outer) conditional loop testing `Flag`
	2 Correct assignment of `Flag` from `GetStat()` **in a loop**
	3 (Inner) loop checking / counting port // Check if `Port` is different to 4
	4 … loop for 3 iterations
	5 a call to `Reset()` **in a loop**
		5

Examiner report missing
9618_s24_qp_22 Question 2

7(a)	Examples include: Module: `IdentifyMember()` Use: Identifies a club member who has expressed an interest in a given class Module: `GetMemberPhoneNumber()` Use: Gets the mobile phone number of a member Module: `CreateMessage()` Use: Generates a text message to a member Module: `SendMessage()` Use: Sends a text message to a member in the waiting list One mark for name **and** use **Note: max 3 marks**	**3**

Examiner report missing
9618_s24_qp_22 Question 7

8(a)	One mark for reason, one for benefit Reason: (Program is) easier to design / implement / test / debug / modify Benefit: Easier to check that **each stage** works as expected	**2**

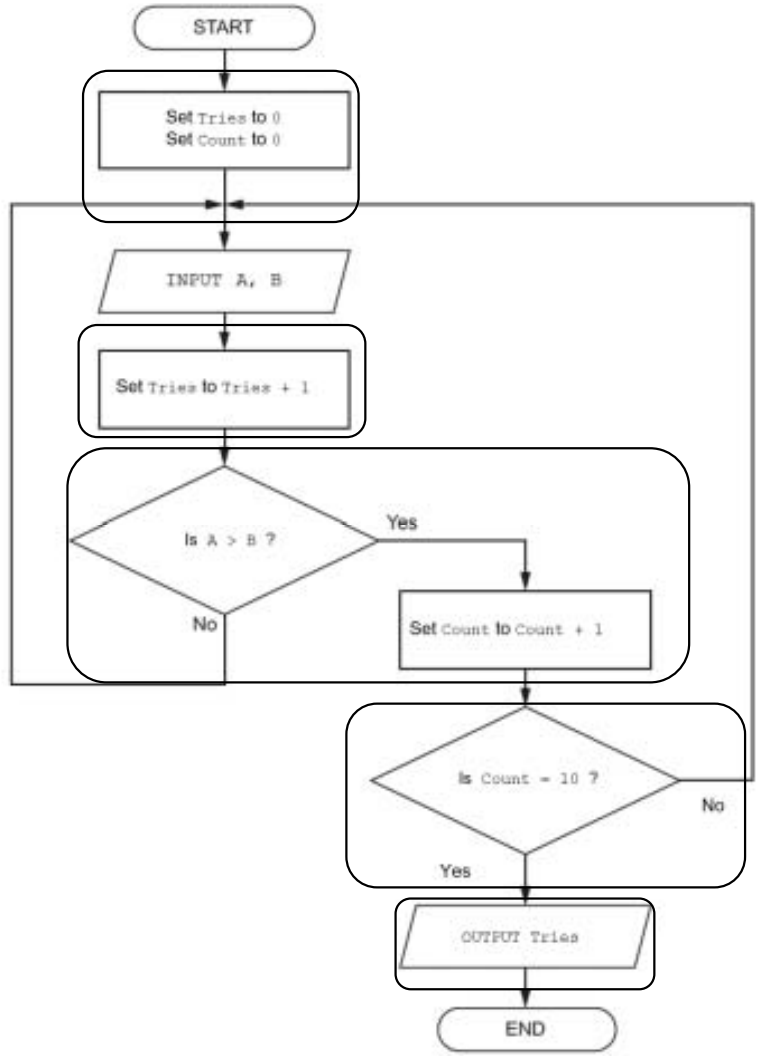
Examiner report missing
9618_s24_qp_22 Question 8

1(a)	<table><tr><th>Assignment statement</th><th>Data type</th></tr><tr><td>A ← LEFT(MyName, 1)</td><td>CHAR / STRING</td></tr><tr><td>B ← Total * 2</td><td>INTEGER / REAL</td></tr><tr><td>C ← INT(ItemCost) / 3</td><td>REAL</td></tr><tr><td>D ← "Odd OR Even"</td><td>STRING</td></tr></table> One mark per row	**4**

1(c)(i)	The names do not reflect / indicate the purpose of the variable // the names are not meaningful	**1**

1(c)(ii)	They make the program more difficult to understand / debug / maintain	**1**

Examiner report missing
9618_s24_qp_21 Question 1

2(a)	 One mark per outlined region: 1 Initialise both counts 2 Increment `Tries` every time a pair is input 3 Compare A > B **and** increment `Count` if TRUE 4 Test for Count = 10 (10th time A > B) – MUST include Yes / No labels 5 If so output `Tries`, otherwise loop	**5**

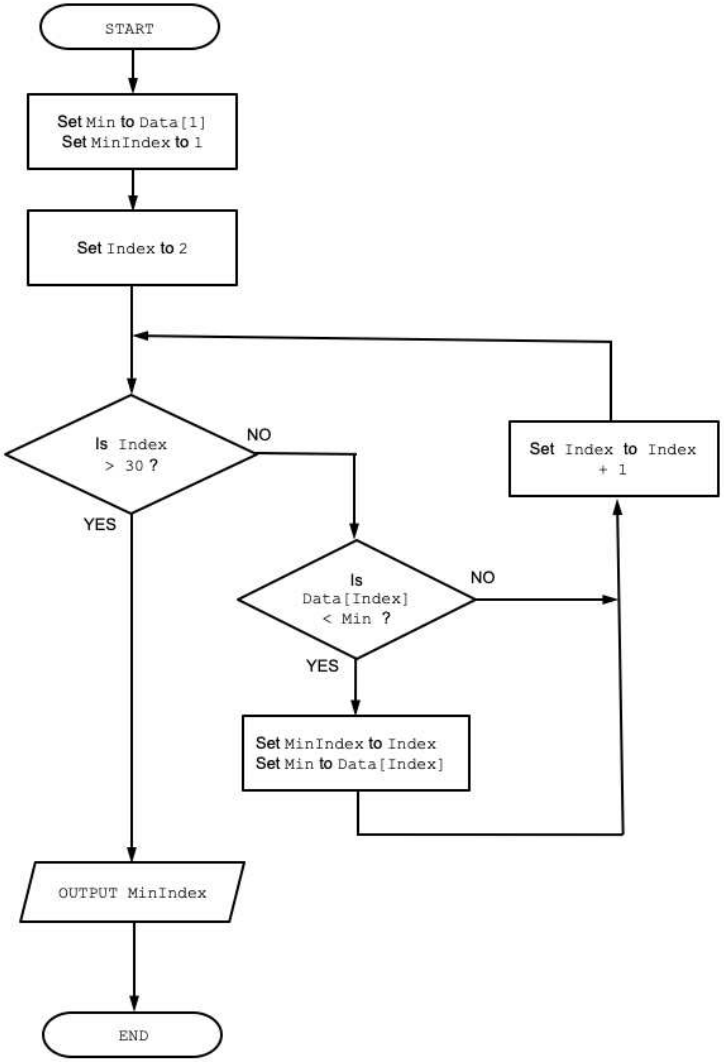
Examiner report missing
9618_s24_qp_21 Question 2

7(a)(i)	To make the solution easier to design / implement / solve	**1**

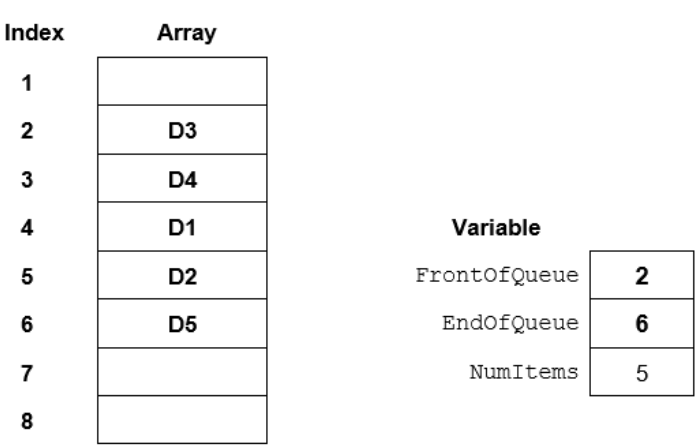
7(a)(ii)	One mark per item **and** justification Item: mobile phone number Justification: to send the text message Item: name Justification: to personalise the text message Item: exercise interest Justification: to determine whether this member would be interested	**3**

7(a)(iii)	Examples include: • Add a member to a list of those interested in the new class • Remove the member from future SMS messages • Read/process Message • Identify who from One mark for each. **Max 2 marks**	**2**

Examiner report missing
9618_s24_qp_21 Question 7

2(a)	 **MP1** Initialise `Min` to first value in `Data` **and** `MinIndex` to 1 **MP2** Loop through 29 more values (or 30 values in total) **MP3** Compare element from `Data[]` with `Min` **MP4** Set new `Min` **AND** save `MinIndex` when element value < `Min` **in a loop** **MP5** Output `MinIndex`	**5**

Examiner report missing
9618_w23_qp_23 Question 2

3(a)	 : **MP1** D3, D4, D1, D2 and D5 in question order or reversed - in any five consecutive locations **MP2** `FoQ` value matches index with D3 **MP3** `EoQ` value matches index with D5	**3**
3(b)	**MP1** If `NumItems` **is / = 8** // **(queue) full** then jump to step 6 **MP2** Increment **`EndOfQueue`** **MP3** If **`EndOfQueue = 9`** then set `EndOfQueue` to **1** **MP4** Increment **`NumItems`** **MP5** Set the **Element** at the index … **MP6** stored in **`EndOfQueue`** to **value/data/item** being added Mark as follows: Steps 1 to 4: One mark for gaps filled as shown Step 5: One mark for 'element' and one mark for other two terms	**6**

Examiner report missing
9618_w23_qp_23 Question 3

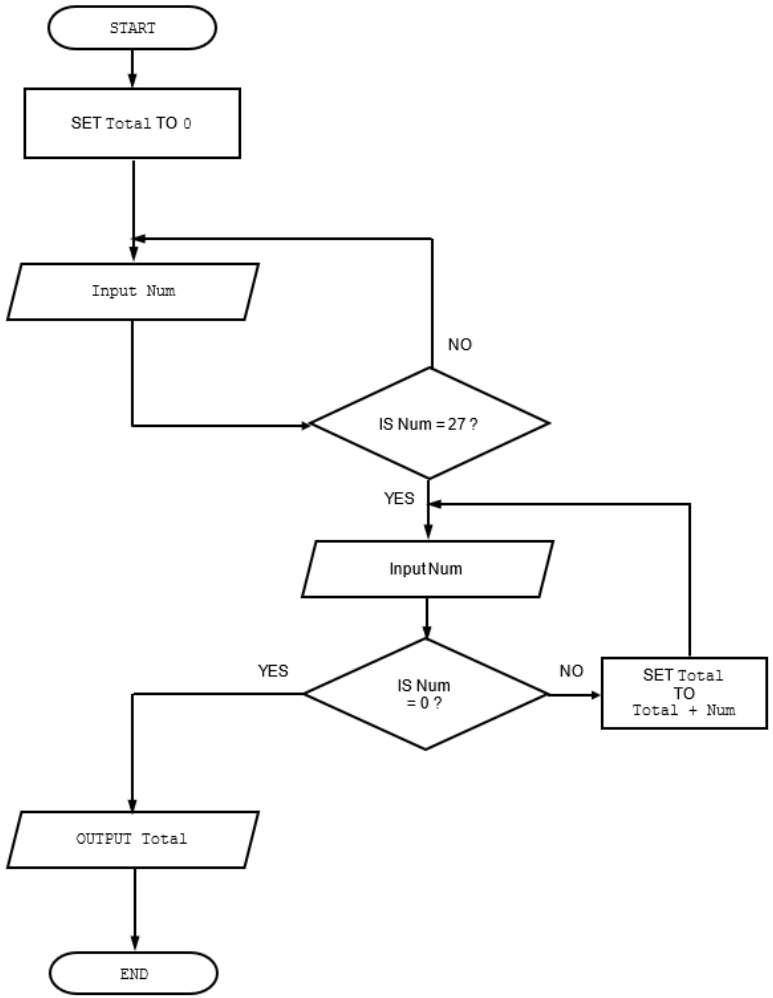
4(a)	``` PROCEDURE RandList() DECLARE Count, BaseNum, ThisNum : INTEGER CONSTANT StepVal = 10 BaseNum ← 0 FOR Count ← 1 TO 25 ThisNum ← BaseNum + INT(RAND(StepVal)) OUTPUT ThisNum BaseNum ← BaseNum + StepVal NEXT Count ENDPROCEDURE ``` **MP1** Procedure heading and ending **MP2** Local loop counter `Count` as integer **MP3** Loop to iterate 25 times or more for each unique number **MP4** 'Attempt' to generate a random number including use of `INT()` **in a loop** **MP5** Ensure that number generated is greater than previous and change 'previous' **MP6** Output random number after an attempt at MP5 **in a loop**	**6**

4(b)	**One** mark for simplified logical expression: **MP1** `Result[x + 1] <= Result[x]` ALTERNATIVE SOLUTION: `Result[x] >= Result[x + 1]`	**1**

Examiner report missing
9618_w23_qp_23 Question 4

1(a)	**One** mark for each row with appropriate variable name and data type **Example value** | **Explanation** | **Variable name** | **Data type** `"Mr Khan"` | The name of the customer | **`CustomerName`** | **`STRING`** `3` | The number of items in the order | **`NumItems`** | **`INTEGER`** `TRUE` | A value to indicate whether this is a new customer | **`NewCustomer`** | **`BOOLEAN`** `15.75` | The deposit paid by the customer | **`Deposit`** | **`REAL`**	**4**

Examiner report missing
9618_w23_qp_22 Question 1

2(a)	Example Solution  **One** mark per point: 1 Initialise `Total` to zero 2 Check for only **first** input of 27 **in a loop** then attempt to sum values 3 Loop until 0 input 4 Sum values **input** (after input of first 27) **in a loop** 5 Output `Total` after a reasonable attempt	**5**

Examiner report missing
9618_w23_qp_22 Question 2

2(a)	**Max 5** marks **MP1** Set total to zero **MP2** Input a number **MP3** Check if number greater than 29 and less than 71 **MP4** … if check is true - add number to total **MP5** Repeat from step 2 99 times // for a total of 100 iterations **MP6** Output the total	**5**

2(b)	**MP1** An iterative construct // a (count-controlled) loop **MP2** A selection construct // an IF statement	**2**

Examiner report missing
9618_w23_qp_21 Question 2

8(a)	``` PROCEDURE SendFile(FileName, DestID : STRING, Port : INTEGER) DECLARE FileData : STRING CONSTANT STX = CHR(02) CONSTANT ETX = CHR(03) OPENFILE FileName FOR READ WHILE NOT EOF(FileName) READFILE FileName, FileData FileData ← STX & DestID & MyID & FileData & ETX CALL Transmit(FileData, Port) ENDWHILE CLOSEFILE FileName CALL Transmit(STX & DestID & MyID & "****" & ETX, Port) ENDPROCEDURE ``` Mark as follows: **MP1** OPEN file in READ mode – using parameter - and subsequently CLOSE **MP2** Conditional loop to `EOF()` **MP3** Use of `READFILE` to get a line from the file **MP4** 'Attempt' to form a message (minimum is `DestID, MyID, FileData`) **MP5** Message formed is completely correct **MP6** Call `Transmit()` with correct MP4 string **in a loop** **MP7** Transmit the "****" message (all parts present) **after the loop**	**7**
8(b)	**Max 2** marks **MP1** Indicates that all the lines of the file have been sent // it is the end of the transmission / file transfer **MP2** So that the receiving program can stop waiting for further data **MP3** The file can be closed / saved	**2**

8(c)(i)	**MP1** A message cannot contain a zero-length data field **MP2** … so a blank line cannot be sent // there is no way to send a blank line	**2**

8(c)(ii)	**MP1** Append a (special) character to the start of the message text **MP2** interpret the new field data as a blank line ALTERNATIVE **MP1** Change the message protocol and use an additional field to act as an indicator **MP2** Interpret the new field data	**2**

Examiner report missing
9618_w23_qp_21 Question 8

1(a)	**Answer** The dimension of the array: **2** The name of the variable used as an array index: **PCount** The number of elements in the array: **100**	**3**

1(d)	One mark for each of rows 2 - 5	**4**

Pseudocode statement	Input	Process	Output
`INPUT MyChoice`	✓		
`OUTPUT FirstName & LastName`		✓	✓
`WRITEFILE OutputFile, TextLine`			✓
`READFILE MyFile, TextLine`	✓		
`Result ← SQRT(NextNum)`		✓	

Examiner report missing
9618_s23_qp_23 Question 1

2(a)	One mark for each underlined part `IF DAYINDEX(MyDOB)  = 5  THEN`	**2**

2(b)(i)	MP1 Value for month is between 1 and 12 (inclusive) MP2 Value of year is <= 2002	**2**

2(b)(ii)	MP1 Reference to month and day MP2 Clear description for a check that the day number matches with a relevant month (Either day matches with month // month matches with day)	**2**

Examiner report missing
9618_s23_qp_23 Question 2

3(a)(i)	6	**1**

3(b)(i)	So that the data may be recovered / restored (the next time the program is run) // the data is permanently saved **/** data is not lost when the program terminates	**1**

3(b)(ii)	**Max 5 marks** MP1 Open the text file in WRITE mode MP2 Check there is a value on the stack MP3 POP value …. MP4 Write value to the text file MP5 Repeat from Step 2 // loop referencing the stack items Alternative solution: Not using POP primitive MP1 Open the text file in WRITE mode MP2 Check there is a value on the stack MP3 Read value from ToS location MP4 Write the value to the text file – Must some attempt at 'the value' NOT 'all the values' MP5 Decrement ToS MP6 Repeat from step 2 // loop referencing the stack items	**5**

Examiner report missing
9618_s23_qp_23 Question 3

6(a)	**Max 7 marks**
	
	`PROCEDURE Select(Start, End : INTEGER)`
	`  DECLARE ThisNum, Total: INTEGER`
	`  DECLARE ThisString : STRING`
	`  DECLARE Char1, Char2 : CHAR`
	
	`  FOR ThisNum ← Start+1 TO End-1`
	`  ThisString ← NUM_TO_STR(ThisNum)`
	`  Char1 ← RIGHT(ThisString, 1)`
	`  Char2 ← LEFT(RIGHT(ThisString, 2), 1)`
	`  Total ← STR_TO_NUM(Char1) + STR_TO_NUM(Char2)`
	`  IF Total = 6 THEN`
	`  OUTPUT ThisString`
	`  ENDIF`
	`  NEXT ThisNum`
	
	`ENDPROCEDURE`
	
	MP1 Procedure heading and ending
	MP2 (Count-controlled) Loop
	MP3 …. with correct range from Start+1 to End-1
	MP4 Convert `ThisNum` (loop counter) to a string
	MP5 Extract the last two/first/second 'character digit(s)' required **in a loop**
	MP6 Extract the second individual 'character digit' required **in a loop**
	MP7 Calculate the sum of the last two digits
	MP8 If sum = 6 then OUTPUT the number (either string or integer) **in a loop**
		7

6(b)	**Max 4 marks** MP1 The function will take two integer parameters - the number and the (required) total MP2 … and return a Boolean OR: `CheckNum(Number,Total : INTEGER)` MP1 `RETURNS BOOLEAN` MP2 Two marks for the advantages: MP3 `CheckNum()` can be called repeatedly as and when required MP4 `CheckNum()` is designed and tested once (then used repeatedly) MP5 Any subsequent change to `CheckNum()` needs to be made once only // is easier to maintain/modify	**4**

Examiner report missing
9618_s23_qp_23 Question 6

7(a)(i)	• To filter out information (that is not necessary to solve the problem) // to include only essential information	**1**

7(a)(ii)	**Required:** Student : Student name / email (address) Loan: Return/Issue date Book: Book title **Not Required:** Student: Home address / DoB / tutor / subject choices Book: Library location / category / author / book title	**2**

7(a)(iii)	**Max 2 marks** Examples could include: • Clear the loan // indicate that the book has been returned // update loan history • Take the student off 'block' // allow the student to borrow further books • Send acknowledgement to the student when book is returned	**2**

Examiner report missing
9618_s23_qp_23 Question 7

8(a)		**8**

```
FUNCTION ChangeSupp(Code1, Code2 : STRING) RETURNS
                                                    INTEGER
  DECLARE Count : INTEGER
  DECLARE ThisLine, ThisCode : STRING

  OPENFILE "Stock.txt" FOR READ
  OPENFILE "NewStock.txt" FOR WRITE
  Count ← 0
  WHILE NOT EOF("Stock.txt")
     READFILE("Stock.txt ", ThisLine) // brackets
                                                    optional
     ThisCode ← MID(ThisLine, 5, 3)
     IF ThisCode = Code1 THEN
        ThisLine ← LEFT(ThisLine, 4) & Code2
                        & RIGHT(ThisLine,
                                  LENGTH(ThisLine) - 7)
        Count ← Count + 1
     ENDIF
     WRITEFILE("NewStock.txt", ThisLine) // brackets
                                                    optional
  ENDWHILE

  CLOSEFILE "NewStock.txt"
  CLOSEFILE "Stock.txt"

RETURN Count
ENDFUNCTION
```

MP1 Open both files, in correct modes, and subsequently close
MP2 Conditional loop until `EOF("Stock.txt")`
MP3 Read a line from `Stock.txt` **AND** extract `ThisCode` **in a loop**
MP4 Test `ThisCode = Code1` AND if true, increment `Count` (must have been Initialised) **in a loop**
MP5 Update `ThisLine` using substring functions and '&' **in a loop**
MP6 completely correct update of `ThisLine` **in a loop**
MP7 Write `ThisLine` to `NewStock.txt` **in a loop**
MP8 Return count after loop

8(c)(i)	**Max 2 marks** MP1 Must ‘calculate’ the count before any item + description output / after the file is read once MP2 Lines to be output have to be stored … MP3 The file has to be read twice	**2**

8(c)(ii)	One mark per point: MP1 Loop through the file calculating the count MP2 Save ‘selected’ items in an array MP3 (After all lines have been read), output the header lines / count MP4 Loop through the array to output each array element	**3**

Examiner report missing
9618_s23_qp_23 Question 8

1(a)(i)	Use of constants	**1**

1(b)	One mark per bullet point: • Indentation • White space • Comments • Sensible / meaningful variable names // use of Camel Case • Capitalised keywords **Note: Max 3 marks**	**3**

Examiner report missing
9618_s23_qp_22 Question 1

2(a)	`MyDOB ← SETDATE(17, 11, 2007)`	**1**

2(c)	One mark per array definition bullet: • A (1D) array containing 7 elements • of type STRING One mark per Step: Step1: Assign value "Sunday" to first element, "Monday" to second element etc. Step2: Use the function `DAYINDEX()` to return / find the day number from `MyDoB` Step3: Use the returned value as the array index / to access the element that contains the name / string Step4: Output the element / name / string **Note: Max 2 for Array definition, Max 4 for steps**	**6**

Examiner report missing
9618_s23_qp_22 Question 2

3(a)(i)	One mark per point: 1 Check that the queue is not full 2 `EoQ` pointer will move to point to location 9 3 Data item Orange will be stored in location referenced by EoQ pointer 4 `EoQ` pointer will move to point to location 0 5 Data item Yellow will be stored in location referenced by EoQ pointer **Note: max 4 marks**	**4**

3(b)	One mark per bullet: 1 Open file in `READ` mode 2 Loop to `EOF()` // read / process all the lines in file 3 Loop will end when return value from `AddToQueue()` is `FALSE` / queue is full 4 Read a line from the file in a loop 5 Pass string to `AddToQueue()` // `AddToQueue()` is executed with line as parameter	**5**

Examiner report missing
9618_s23_qp_22 Question 3

7(a)(i)	Correct answers include: • Information: customer name Justification: to personalise / address the email • Information: email address Justification: so that the email can be delivered correctly • Information: product category preference Justification: to check that the customer would be interested in the product • Information: contact preference Justification: to check that the customer will accept contact via email • Information: order history Justification: to send details of product similar to that already bought // to identify frequent shoppers • Information: new product information Justification: to include information about the new product in the email One mark for each piece of information **and** matching justification **Note: Max 3 marks**	**3**

7(a)(ii)	One mark for each piece of information **and** matching justification: • postal address Justification: sending an email, not a letter • payment details Justification: Nothing being bought / sold at this stage • order history Justification: Customer preference used to decide if new product is relevant **Note: Max 2 marks**	**2**

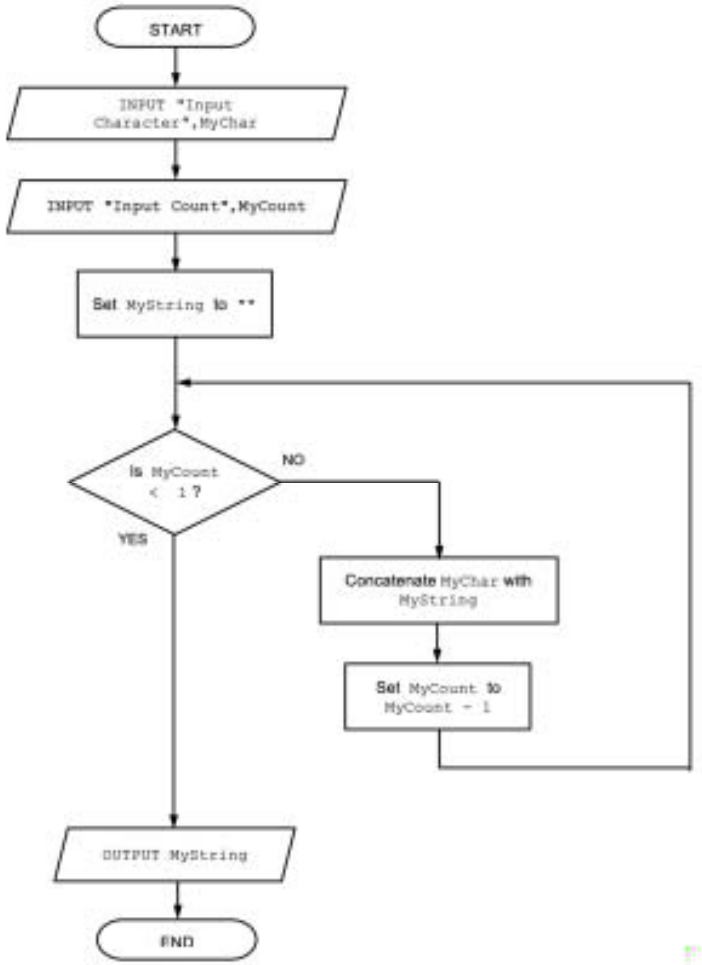
Examiner report missing
9618_s23_qp_22 Question 7

1(a)	For example: Could set a breakpoint to stop the program at a particular point / instruction then the value of variables could be checked using a report/watch window while single stepping can be used to execute one statement/line at a time. Marks available as follows: • One mark for each underlined term • One mark for an explanation of each term **Note: max 4 marks**	**4**

1(c)(i)	<table><tr><th></th><th>Expression</th><th>Evaluation</th></tr><tr><td>1</td><td><code>(Points > 99) OR Active</code></td><td>TRUE</td></tr><tr><td>2</td><td><code>(Points MOD 2 = 0) OR Exempt</code></td><td>FALSE</td></tr><tr><td>3</td><td><code>(Points <= 75) AND (Active OR Exempt)</code></td><td>TRUE</td></tr><tr><td>4</td><td><code>(Active OR NOT Active) AND NOT Exempt</code></td><td>TRUE</td></tr></table> One mark for any two rows correct Two marks for all rows correct	**2**

1(c)(ii)	`NOT Exempt`	**1**

Examiner report missing
9618_s23_qp_21 Question 1

2(a)	 One mark per point: 1 Both prompts 2 Both inputs using correct identifiers as given in question, `MyChar` and `MyCount` 3 Initialise `MyString` to empty string **and** subsequent output **after loop** 4 Loop `MyCount` iterations including decrement `MyCount` 5 Use of Concatenate or equivalent pseudocode statement **inside loop** **Max 4 Marks**	**4**

Examiner report missing
9618_s23_qp_21 Question 2

3(a)(i)	One mark for structure Structure: Record One mark for each point Advantage: • A set of data / all data related to one customer • of different types • is held under a single identifier/entity	**4**

3(b)	One mark for reference to each: 1 Reference to the use of constants or variables for the two threshold values of 10 and 100 // Input amount spent (by customer and store in a numeric variable) 2 Work out **one** band that amount maps to 3 Work out **all** bands that amount maps to 4 Calculate rounded value of amount / whole number part of amount 5 Calculate the points by multiplying the (rounded) amount by the appropriate value for appropriate band /all bands 6 Output the number of points **Note: Max 5 from available points**	**5**

Examiner report missing
9618_s23_qp_21 Question 3

7(a)	Examples include: Module: `GetOverdueLoan()` Use: Identifies an overdue book Module: `IdentifyStudent()` Use: Identifies a student (with an overdue book)1 Module: `GetStudentEmail()` Use: Gets the email address of a student with an overdue book Module: `CreateEmail()` Use: Generates an email to a student with an overdue book Module: `SendEmail()` Use: Sends an email to a student with an overdue book One mark for name **and** use **Note: Max 3 marks**	**3**

Examiner report missing
9618_s23_qp_21 Question 7

2	One mark for name **and** two marks for use (**Max 3 in total**): Examples include: Module: `SelectCharity()` Use: Allows the user to choose a particular charity Module: `SpecifyAmountAndType()` Use: Allows the user to specify a single or regular payment Module: `MakePayment()` Use: Make payment to the charity Module: `ValidatePayment()` Use: Validate payment details (by accessing bank computer) Module: `AddBankAccountDetails() / AddPaymentDetails()` Use: Allows the user to add bank account information that donation to be taken from Module: `AddDonorDetails()` Use: Allows user to add details such as name and contact details	**3**

Examiner report missing
9618_w22_qp_23 Question 2

3	One mark per point, for example: 1 Declare two (`REAL`) variables for the two sum values **AND** initialise both to zero 2 Prompt **AND** Input a number 3 If number greater than zero add to positive sum and If number less than zero add to negative sum 4 Repeat from step 2 if number not zero 5 After loop the Output `SumPos` and `SumNeg`	**5**

Examiner report missing
9618_w22_qp_23 Question 3

5(a)(i)	One mark per point:
	
	1 Procedure heading and ending including four parameters...
	2 ...and use of `BYREF` for the three extracted values
	3 Extract and assign `SID`
	4 Extract and assign `SDesc`
	5 Calculation of length of `SCost` (remainder of string)
	6 Extract and assign `SCost` following an attempt at MP5
	
	`PROCEDURE UnPack(BYVAL TLine : STRING, BYREF SID, SDesc, SCost : STRING)`
	`  SID ← LEFT(TLine, 5)`
	`  SDesc ← MID(TLine, 6, 32)`
	`  SCost ← RIGHT(TLine, LENGTH(TLine) - 37)`
	`ENDPROCEDURE`
		6

5(c)	One mark for reason One mark for each example (**Max 2**) Reason: • Makes the code easier to understand // Describes the purpose of the identifier // Makes the code easier to debug/test/maintain Further examples include: • White space • Indentation • Keywords in capitals • Comments • Local variables // parameters	**3**

Examiner report missing
9618_w22_qp_23 Question 5

7(a)	One mark per point (**Max 8**):
	
	1 Declaration and initialisation of local integer for `Count`
	2 Appropriate prompt and two inputs
	3 (Conditional) loop while error number input is in range // error code 999 reached
	4 …and not end of array
	5 Check if this `ErrCode` needs to be output **in a loop**
	6 if so check for blank error text **in a loop**
	7 Output in both cases
	8 ….and increment count **in a loop**
	9 OUTPUT of header and summary including count
	
	`PROCEDURE OutputRange()`
	`  DECLARE First, Last, Count, Index, ThisErr : INTEGER`
	`  DECLARE ThisMess : STRING`
	`  DECLARE PastLast: BOOLEAN`
	
	`  Count ← 0`
	`  Index ← 1`
	`  PastLast ← FALSE`
	
	`  OUTPUT "Please input first error number: "`
	`  INPUT First`
	`  OUTPUT "Please input last error number: "`
	`  INPUT Last`
	
	`  OUTPUT "List of error numbers from ", First, " to ", Last`
	
	`  WHILE Index < 501 AND NOT PastLast`
	`  ThisErr ← ErrCode[Index]`
	`  IF ThisErr > Last THEN`
	`  PastLast ← TRUE`
	`  ELSE`
	`  IF ThisErr >= First THEN`
	`  ThisMess ← ErrText[Index]`
	`  IF ThisMess = "" THEN`
	`  ThisMess ← "Error Text Missing"`
	`  ENDIF`
	`  OUTPUT ThisErr, " : ", ThisMess`
	`  Count ← Count + 1`
	`  ENDIF`
	`  ENDIF`
	`  Index ← Index + 1`
	`  ENDWHILE`
	
	`  OUTPUT Count, " error numbers output"`
	
	`ENDPROCEDURE`
		8

7(b)(i)	One mark per point:
	
	1 (Conditional) loop terminating when item added **OR** end of array reached
	2 Test for unused element **in a loop**
	3 Assignment of values to arrays // save index of first blank location and assign after loop
	4 Set loop termination if empty element found in a loop
	5 Call `SortArrays()` **once**
	6 Calculation of remaining unused elements **and** return Integer value (for both cases)
	
	`FUNCTION AddError(ErrNum : INTEGER, ErrMess : STRING)`
	`RETURNS INTEGER`
	`  DECLARE Index, Remaining : INTEGER`
	`  CONSTANT Unused = 999`
	
	`  Index ← 1`
	`  Remaining ← -1`
	
	`  REPEAT`
	`    IF ErrCode[Index] = Unused THEN`
	`      ErrCode[Index] ← ErrNum`
	`      ErrText[Index] ← ErrMess`
	`      CALL SortArrays()`
	`      Remaining ← 500 - Index`
	`    ENDIF`
	`    Index ← Index + 1`
	`  UNTIL Remaining <> -1 OR Index > 500`
	
	`  RETURN Remaining`
	
	`ENDFUNCTION`
		6

7(b)(ii)	One mark per point (**Max 3**): 1. Loop through 500 elements (while error number not found) 2. Compare `ErrCode` for current element with the error number 3. If same, set element value to 999 (and terminate loop) 4. ... and call `SortArrays()` (to move 999 to the end) – once only	**3**

Examiner report missing
9618_w22_qp_23 Question 7

1(a)(i)	One mark for each point (**Max 2**): • When a task which is repeated / reused / performed in several places • When a part of an algorithm performs a specific task • Reduces complexity of program / program is simplified // subroutine already available • Testing / debugging / maintenance is easier	**2**

Examiner report missing
9618_w22_qp_22 Question 1

2(a)(i)	One mark per step (or equivalent): 1 Open file in APPEND mode (and subsequent Close) 2 Prompt and Input a student name and mark 3 If mark greater than or equal to 20 jump to step 5 4 Write only the name to the file 5 Repeat from Step 2 for 35 times / the number of students	**5**

Examiner report missing
9618_w22_qp_22 Question 2

4(a)	One mark per point:
	
	1 Input `UserID` **and** use of `GetAverage()` **and** assignment
	2 Initialisation of `Total` to zero **and** `Index` to 4
	3 Conditional loop with `Index` from 4 to 6
	4 Assignment of `Last` from element `SameMonth[Index]` **in a loop**
	5 Structure: `IF...THEN...ELSE...ENDIF` **in a loop**
	6 Correct assignments **and** final call to `Update()` **after the loop**
	
	`INPUT UserID`
	`Average ← GetAverage(UserID)`
	`Total ← 0`
	`Index ← 4`
	
	`WHILE Index < 7 // REPEAT`
	`  Last ← SameMonth[Index]`
	`  IF Average > Last THEN`
	`  Total ← Total + Average`
	`  ELSE`
	`  Total ← Total + Last`
	`  ENDIF`
	`  Index ← Index + 1`
	`ENDWHILE // UNTIL Index = 7`
	
	`CALL Update(UserID, Total)`
	
	<u>Alternative solution using FOR loop</u>:
	
	One mark per point FOR loop solution:
	1 Input `UserID` **and** use of `GetAverage()` **and** assignment
	2 Initialisation of `Total` to zero
	3 loop `Index` from 4 to 6
	4 Assignment of `Last` from array `SameMonth` **in a loop**
	5 Comparison `IF...THEN...ELSE...ENDIF` **in a loop**
	6 Appropriate assignments **in a loop AND** final call to `Update()` **after the loop**
	
	`INPUT UserID`
	`Average ← GetAverage(UserID)`
	`Total ← 0`
	`FOR Index ← 4 TO 6`
	`  Last ← SameMonth[Index]`
	`  IF Average > Last THEN`
	`  Total ← Total + Average`
	`  ELSE`
	`  Total ← Total + Last`
	`  ENDIF`
	`NEXT Index`
	
	`CALL Update(UserID, Total)`
		6

Examiner report missing
9618_w22_qp_22 Question 4

2(a)	One mark for Explanation: • Abstraction is used to filter out information / data that is not necessary for the task Or the opposite: • To keep only information / data that is necessary for the task One mark for each **TWO** data items (not dependent on 'Explanation'): Items include: • Car details: ID, Car Registration, car type etc • Customer details: ID, name, address, licence details etc • Start date (of hire) • Return date / Number of days (of hire) • Cost of hire	**3**

2(b)	One mark for each (**Max 2**) Examples include: 1 Input customer details 2 Input car details 3 Input payment details 4 Create hire / start hire 5 Return car / end hire 6 Change / check car status (hired / available / written-off) 7 Cancel hire 8 Process payment / calculate hire cost	**2**

Examiner report missing
9618_w22_qp_21 Question 2

3	One mark per point (**Max 5**): 1 Declare a variable / an integer `Max` 2 Assign value of first / any element to `Max` 3 Set up a loop to repeat 200 times / from start to end of array 4 Use the loop counter as the array index 5 If value of current element is greater than `Max`... 6 ...then assign value to `Max` 7 After the loop, Output `Max`	**5**

Examiner report missing
9618_w22_qp_21 Question 3

1(a)(i)	One from: • The program obeys the rules / grammar of the programming language used • The program will run // it can be compiled / interpreted • Accept by example. e.g. 'no mis-spelt keywords' / 'all brackets match'	**1**

1(c)(i)	<table><tr><th>Information</th><th>Essential</th><th>Not essential</th></tr><tr><td>Departure time</td><td>✓</td><td></td></tr><tr><td>Flight Number</td><td></td><td>✓</td></tr><tr><td>Departure airport</td><td>✓</td><td></td></tr><tr><td>Aircraft type</td><td></td><td>✓</td></tr><tr><td>Ticket price</td><td>✓</td><td></td></tr><tr><td>Number of seats in aircraft</td><td></td><td>✓</td></tr></table> One mark for two rows correct Two mark for four rows correct Three mark for all rows correct	**3**

1(c)(ii)	One mark for technique and one for benefit, **Max 1** mark for 'Benefit' Technique: Abstraction Benefit: • The solution is simplified so easier / quicker to design / implement • The system is tailored to the need of the user	**2**

1(c)(iii)	Answers include: • Destination / arrival airport • Arrival time / flight duration • Date of flight • Seat number • Seat availability **Max 2** marks	**2**

Examiner report missing
9618_w21_qp_23 Question 1

2(a)	One mark for reference to: 1 The use a variable as an index to the array 2 A loop to iterate through the array 3 An Inner loop (with a reducing range) 4 Test if current element is greater than next element 5 if so then swap elements 6 Description of swap 7 Attempt at efficient algorithm **Max 6** marks	**6**

2(b)	`Count ← 1`
	`Flag ← FALSE`
	`WHILE Flag = FALSE AND Count <= 5`
	`  CALL ReBoot()`
	`  Count ← Count + 1`
	`  Flag ← Check()`
	`ENDWHILE`
	
	`IF Flag = FALSE THEN`
	`  CALL Alert(27)`
	`ENDIF`
	
	One mark per point:
	
	1 Initialisation of `Count` **AND** `Flag`
	2 `WHILE ... ENDWHILE // REPEAT ... UNTIL` loop
	3 ... including both conditions
	4 Call `ReBoot()` **AND** increment `Count` **inside the loop**
	5 Assign return value from `Check()` to `Flag` **inside the loop**
	6 Final test of `Flag` **AND** call to `Alert(27)` **not in a loop**
		6

Examiner report missing
9618_w21_qp_23 Question 2

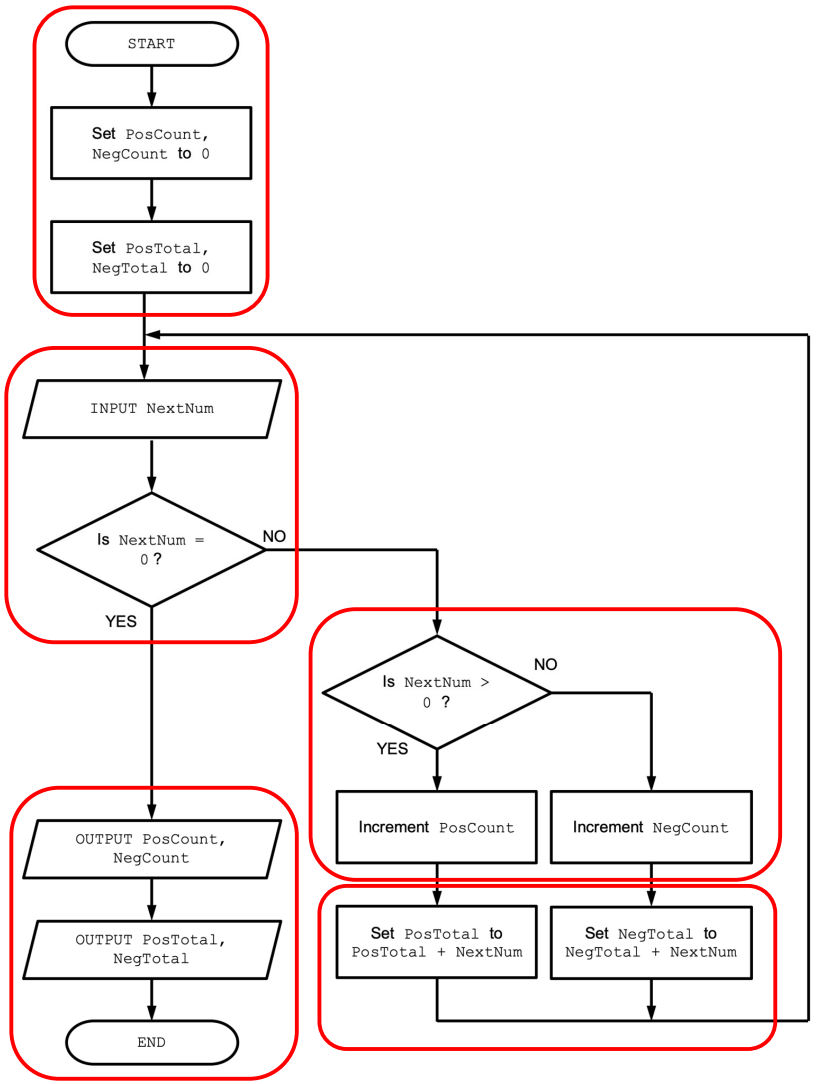
3(a)(i)	One mark per point: • `EoQ` pointer will move to point to location 4 // incremented `EoQ` (by 1) • Data value "Octopus" will be stored in location pointed to be `EoQ` / location 4	**2**

3(c)	One mark per point: 1 If incremented `EoQ` = `FoQ` then error condition: queue is full 2 Increment the `EoQ` 3 Manage wrap-around	**3**

Examiner report missing
9618_w21_qp_23 Question 3

1(a)	The process involves: 1 Breaking down a problem / task into sub problems / steps / smaller parts 2 In order to explain / understand // easier to solve the problem 3 Leading to the concept of program modules // assigning problem parts to teams **Max 2**	**2**

Examiner report missing
9618_w21_qp_22 Question 1

2(a)	 One mark for each outlined group. Note: • Sum and increment steps (bottom-right rectangles) may be in reverse order in which case sum group will have two output lines • Max 4 for non-working solution	**5**

Examiner report missing
9618_w21_qp_22 Question 2

4(a)		**5**

	Answer
A line number containing a variable being incremented	19 / 21 / 23
The type of loop structure	pre-condition
The number of functions used	3
The number of parameters passed to function `STR_TO_NUM()`	1
The name of a procedure other than `Check()`	Result

4(b)	One mark per point: • Meaningful variable names • Indentation / white space / blank lines • Capitalisation of keywords	**3**

Examiner report missing
9618_w21_qp_22 Question 4

6(a)	```
	FUNCTION FirstRowSet() RETURNS INTEGER
	  DECLARE Row, Col : INTEGER
	  DECLARE Found : BOOLEAN
	
	  // array is 1280 × 800
	  Row ← 1
	
	  Found ← FALSE
	  WHILE Row <= 800 AND Found = FALSE // top to bottom
	  Col ← 1
	  WHILE Col <= 1280 AND Found = FALSE // left to right
	  IF Screen[Row,Col] = 1 THEN
	  Found ← TRUE // end function as soon as first
	  // found
	  ENDIF
	  Col ← Col + 1
	  ENDWHILE
	  Row ← Row + 1
	  ENDWHILE
	
	  IF Found = FALSE THEN // nothing found
	  Row ← 0
	  ENDIF
	  RETURN Row - 1
	ENDFUNCTION
	``
	
	Mark as follows:
	
	1 Function heading and ending **and** return type
	2 (Conditional) outer loop 1 to 800 (row)
	3 (Conditional) inner loop 1 to 1280 // 1280 to 1 (column)
	4 Reference `Screen` element and test for = 1 // <> 0
	5 and if true save row number **and** exit loops
	6 Increment index variables in both inner **and** outer loop
	7 Return Row number or −1, following a reasonable attempt
		7

6(c)(i)	One mark for: • Parameter(s) need to be passed to the module to identify the type of search • Search algorithm is controlled by (global) variables / parameters Alternative: • The search algorithms from the original modules are included in the new module • The new module needs to return / store the four values (the results of the four searches)	**2**

6(c)(ii)	One mark for advantage and one for disadvantage: Advantage: (max 1) • Only have to change one module if specification changes • Less repetitive code / fewer lines of code • Aids re-usability Disadvantage: (max 1) • Single module more complex / more error prone / more difficult to debug ... • Single module cannot be split among programmers / teams **Max 2**	**2**

Examiner report missing
9618_w21_qp_22 Question 6

1(a)(i)	One from: • The program obeys the rules / grammar of the programming language used • The program will run // it can be compiled / interpreted • Accept by example. e.g. 'no mis-spelt keywords' / 'all brackets match'	**1**

1(c)(i)	<table><tr><th>Information</th><th>Essential</th><th>Not essential</th></tr><tr><td>Departure time</td><td>✓</td><td></td></tr><tr><td>Flight Number</td><td></td><td>✓</td></tr><tr><td>Departure airport</td><td>✓</td><td></td></tr><tr><td>Aircraft type</td><td></td><td>✓</td></tr><tr><td>Ticket price</td><td>✓</td><td></td></tr><tr><td>Number of seats in aircraft</td><td></td><td>✓</td></tr></table> One mark for two rows correct Two mark for four rows correct Three mark for all rows correct	**3**

1(c)(ii)	One mark for technique and one for benefit, **Max 1** mark for 'Benefit' Technique: Abstraction Benefit: • The solution is simplified so easier / quicker to design / implement • The system is tailored to the need of the user	**2**

1(c)(iii)	Answers include: • Destination / arrival airport • Arrival time / flight duration • Date of flight • Seat number • Seat availability **Max 2** marks	**2**

Examiner report missing
9618_w21_qp_21 Question 1

2(a)	One mark for reference to: 1 The use a variable as an index to the array 2 A loop to iterate through the array 3 An Inner loop (with a reducing range) 4 Test if current element is greater than next element 5 if so then swap elements 6 Description of swap 7 Attempt at efficient algorithm **Max 6** marks	**6**

2(b)	`Count ← 1`
	`Flag ← FALSE`
	`WHILE Flag = FALSE AND Count <= 5`
	`  CALL ReBoot()`
	`  Count ← Count + 1`
	`  Flag ← Check()`
	`ENDWHILE`
	
	`IF Flag = FALSE THEN`
	`  CALL Alert(27)`
	`ENDIF`
	
	One mark per point:
	
	1 Initialisation of `Count` **AND** `Flag`
	2 `WHILE ... ENDWHILE // REPEAT ... UNTIL` loop
	3 ... including both conditions
	4 Call `ReBoot()` **AND** increment `Count` **inside the loop**
	5 Assign return value from `Check()` to `Flag` **inside the loop**
	6 Final test of `Flag` **AND** call to `Alert(27)` **not in a loop**
		6

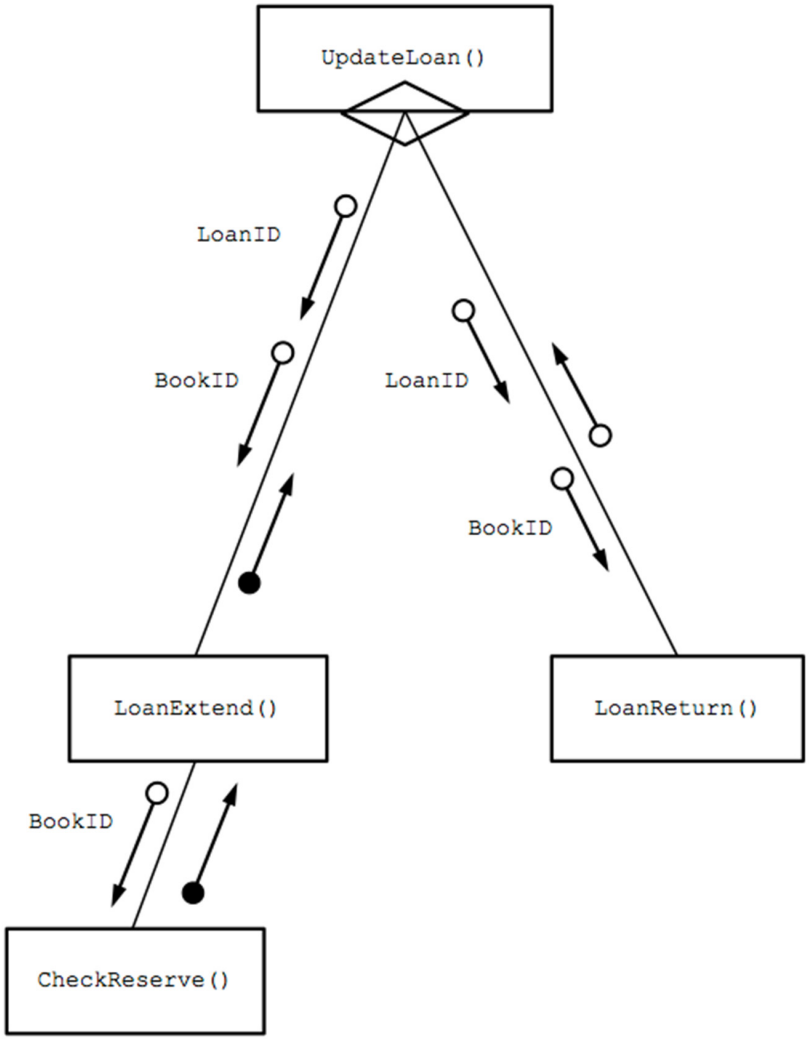
Examiner report missing
9618_w21_qp_21 Question 2

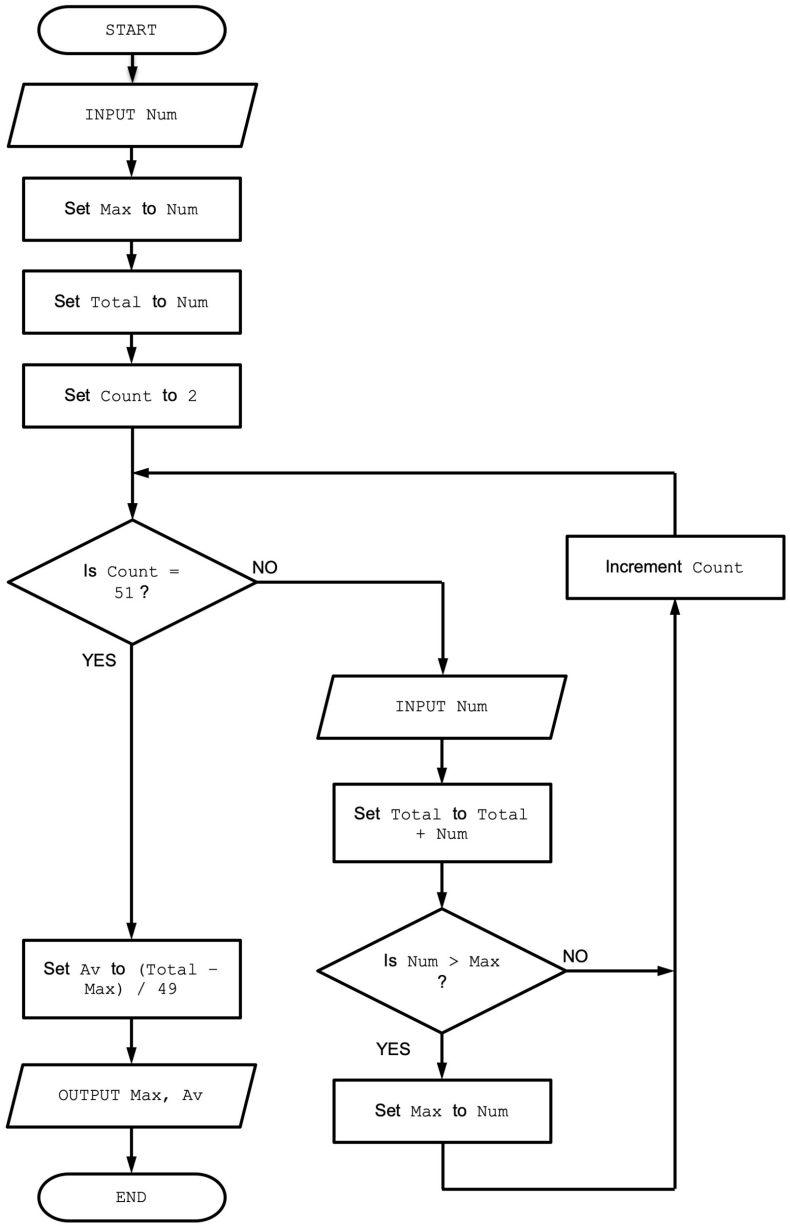
1(a)		**4**

Example value	Explanation	Variable name	Data type
`"Wong"`	The preferred name of the member joining the football club	**`MemberName`**	**`STRING`**
`FALSE`	A value to indicate whether an existing member of the club lives at the same address	**`FamilyMember`**	**`BOOLEAN`**
`19/02/1983`	When the member joined the football club	**`StartDate`**	**`DATE`**
`1345`	The number of points a member has earned. Members of the club earn points for different activities.	**`Points`**	**`INTEGER`**

One mark for each appropriate variable name plus data type

Examiner report missing
9618_s21_qp_23 Question 1

2(a)	 One mark for: 1 All four boxes correctly labelled and positioned 2 Selection diamond (only on `UpdateLoan` and no iteration arrows) 3 Parameters to and from `LoanExtend()` 4 Parameters to and from `CheckReserve()` 5 Parameters to and from `LoanReturn()`	**5**

2(c)	Example solution:  One mark for each functional group as listed below:	**6**

Examiner report missing
9618_s21_qp_23 Question 2

7(a)	
	``
	FUNCTION GetStart (WordNum : INTEGER) RETURNS INTEGER
	  DECLARE Index, ThisPos, NumFound : INTEGER
	  DECLARE ThisChar : Char
	
	  CONSTANT SPACECHAR = ' '
	
	  Index ← -1
	  Numfound ← 0
	  ThisPos ← 1
	
	  IF WordNum = 1 THEN // if looking for word 1...
	     Index ← 1 // Word 1 always starts at index
	                      // position 1
	  ELSE
	                       // Otherwise start counting spaces...
	     WHILE ThisPos <= LENGTH(FNString) AND Index = -1
	        ThisChar ← MID(FNString, ThisPos, 1)
	        IF ThisChar = SPACECHAR THEN
	           NumFound ← NumFound + 1
	           IF NumFound = WordNum - 1 THEN
	              Index ← ThisPos + 1 // the start of the
	                                  // required word
	           ENDIF
	        ENDIF
	        ThisPos ← ThisPos + 1
	     ENDWHILE
	  ENDIF
	RETURN Index
	
	ENDFUNCTION
	``
	
	1 mark for each of the following:
	
	1 Function heading, including return type and function end
	2 Loop counting spaces until word found or end of `FNString`
	3 extract a character from `FNString` **in a loop**
	4 compare with `SPACECHAR` **and** increment count if equal **in a loop**
	5 compare count with `WordNum - 1` (depending on initialisation value) **in a loop**
	6 if equal then set flag or `Index` to `ThisPos + 1` **in a loop**
	7 Return Index (correctly in all cases / following a reasonable attempt)
	8 Works for special case when looking for word 1
	
	Note: Max 7 marks
		7

7(b)	Marks awarded for any reference to each of the following steps of the algorithm: 1 Mention of variable for use as array index 2 Use of a loop (to check through the array) 3 If word is the same as the current array element then return `FALSE` / set flag 4 If word not already in array, loop to find unused element *(second loop)* 5 Store word in unused element and return `TRUE`, otherwise return `FALSE` VARIATION: 1 Mention of variable for use as array index 2 Use of a loop (to check through the array) 3 Save index of (first) unused element found 4 If word is the same as the current array element then return `FALSE` / set flag 5 If word not already in array **and** unused element available, store word in unused element and return `TRUE` otherwise return `FALSE` **Note: Max 4 marks**	**4**

Examiner report missing
9618_s21_qp_23 Question 7

<table>
<tr><td>2(a)(i)</td><td>
<table>
<tr><td>The number of transitions that result in a different state</td><td>3</td></tr>
<tr><td>The number of transitions with associated outputs</td><td>2</td></tr>
<tr><td>The label that should replace ‘X’</td><td>Start</td></tr>
<tr><td>The final or halting state</td><td>S3</td></tr>
</table>
One mark per row
</td><td>4</td></tr>
<tr><td>2(b)(i)</td><td>Answers include:

• User ID / Username
• Book ID
• Date of loan / return date

One mark for 1 correct
Two marks for all 3 correct

Note: Max 2 marks</td><td>2</td></tr>
<tr><td>2(b)(ii)</td><td>Many examples but must be data that is NOT required for a loan, but which COULD be required somewhere by the library system.

Note: must be data relating to users, books or loans

Answers include:
• Users name / address / phone number / DOB
• Book title / author / publisher / library rack number / ISBN number / price
• Date of loan / return date (if not already given in part (i))
• The length of the loan (assumed to be the same for all books)</td><td>1</td></tr>
</table>

2(b)(iii)	Many examples including: • Create loan / borrow book • Return book • Send letter / email / contact a user ref an overdue book • View the loan history for a given book • View the loan history for a given user One mark for each **Note: Max 2 marks**	**2**

Examiner report missing
9618_s21_qp_22 Question 2

4	Marks awarded for a description of each of the following steps of the algorithm: 1 Reference variables for `Count` of students and `Total` marks 2 Loop through all students (`Count`) 3 Input individual mark **(in loop)** 4 Compare mark with threshold / boundary values to determine grade **(in loop)** 5 Output the grade for a student **(in loop)** 6 Maintain a `Total` (and `Count` if required) **(in loop)** 7 Calculate average by dividing `Total` by `Count` **and** Output (**after loop***)* **Note: Max 6 marks**	**6**

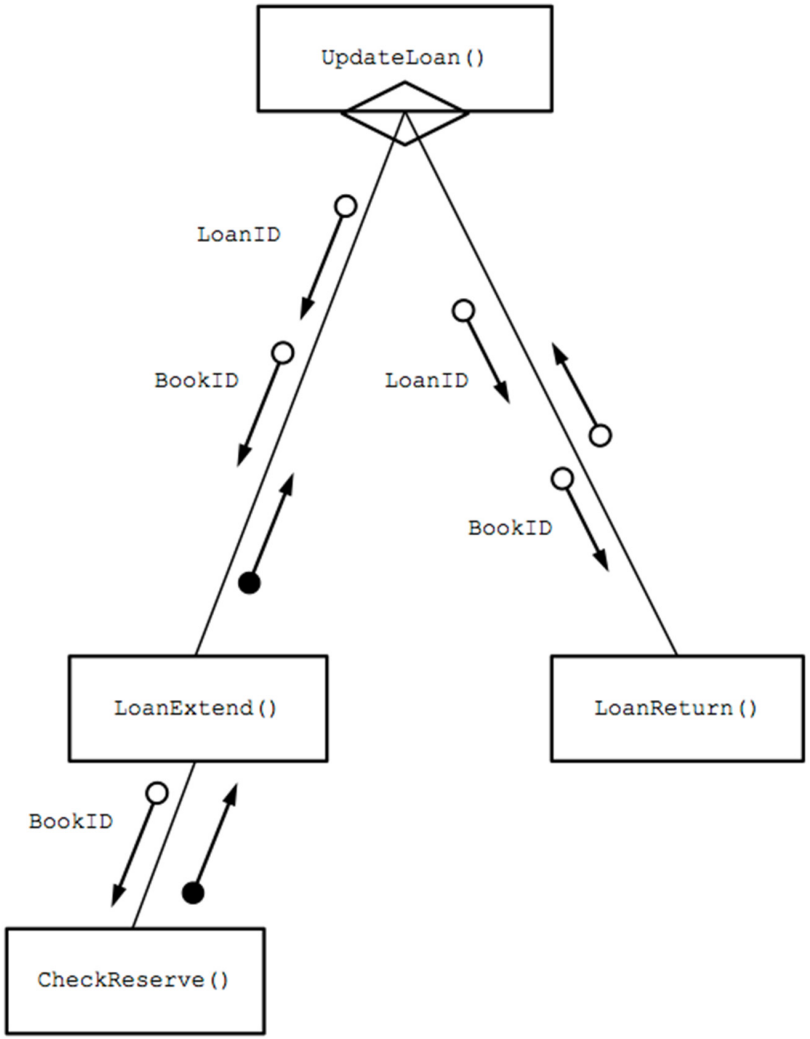
Examiner report missing
9618_s21_qp_22 Question 4

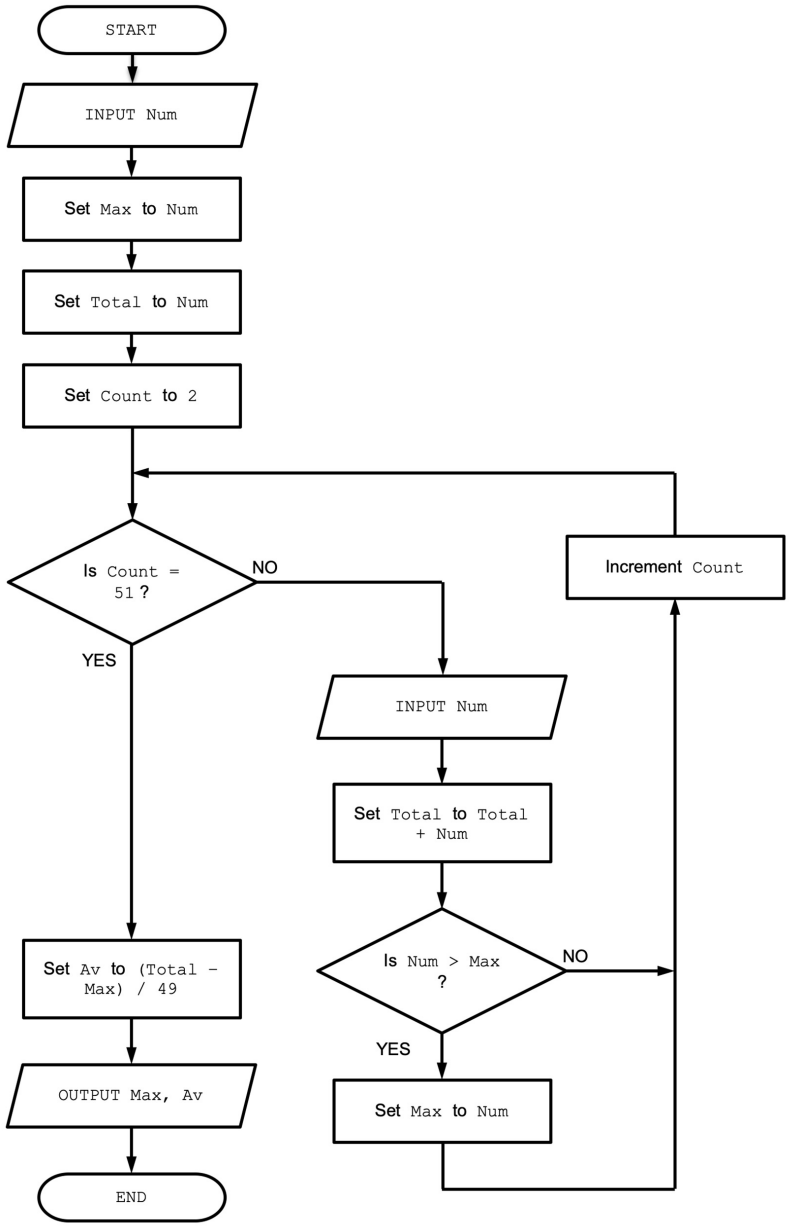
1(a)		**4**

Example value	Explanation	Variable name	Data type
`"Wong"`	The preferred name of the member joining the football club	**`MemberName`**	**`STRING`**
`FALSE`	A value to indicate whether an existing member of the club lives at the same address	**`FamilyMember`**	**`BOOLEAN`**
`19/02/1983`	When the member joined the football club	**`StartDate`**	**`DATE`**
`1345`	The number of points a member has earned. Members of the club earn points for different activities.	**`Points`**	**`INTEGER`**

One mark for each appropriate variable name plus data type

Examiner report missing
9618_s21_qp_21 Question 1

2(a)	 One mark for: 1 All four boxes correctly labelled and positioned 2 Selection diamond (only on `UpdateLoan` and no iteration arrows) 3 Parameters to and from `LoanExtend()` 4 Parameters to and from `CheckReserve()` 5 Parameters to and from `LoanReturn()`	**5**

2(c)	Example solution:  One mark for each functional group as listed below:	**6**

Examiner report missing
9618_s21_qp_21 Question 2

7(a)	
	``
	FUNCTION GetStart (WordNum : INTEGER) RETURNS INTEGER
	  DECLARE Index, ThisPos, NumFound : INTEGER
	  DECLARE ThisChar : Char
	
	  CONSTANT SPACECHAR = ' '
	
	  Index ← -1
	  Numfound ← 0
	  ThisPos ← 1
	
	  IF WordNum = 1 THEN // if looking for word 1...
	     Index ← 1 // Word 1 always starts at index
	                      // position 1
	  ELSE
	                       // Otherwise start counting spaces...
	     WHILE ThisPos <= LENGTH(FNString) AND Index = -1
	        ThisChar ← MID(FNString, ThisPos, 1)
	        IF ThisChar = SPACECHAR THEN
	           NumFound ← NumFound + 1
	           IF NumFound = WordNum - 1 THEN
	              Index ← ThisPos + 1 // the start of the
	                                  // required word
	           ENDIF
	        ENDIF
	        ThisPos ← ThisPos + 1
	     ENDWHILE
	  ENDIF
	RETURN Index
	
	ENDFUNCTION
	``
	
	1 mark for each of the following:
	
	1 Function heading, including return type and function end
	2 Loop counting spaces until word found or end of `FNString`
	3 extract a character from `FNString` **in a loop**
	4 compare with `SPACECHAR` **and** increment count if equal **in a loop**
	5 compare count with `WordNum - 1` (depending on initialisation value) **in a loop**
	6 if equal then set flag or `Index` to `ThisPos + 1` **in a loop**
	7 Return Index (correctly in all cases / following a reasonable attempt)
	8 Works for special case when looking for word 1
	
	Note: Max 7 marks
		7

7(b)	Marks awarded for any reference to each of the following steps of the algorithm: 1 Mention of variable for use as array index 2 Use of a loop (to check through the array) 3 If word is the same as the current array element then return `FALSE` / set flag 4 If word not already in array, loop to find unused element *(second loop)* 5 Store word in unused element and return `TRUE`, otherwise return `FALSE` VARIATION: 1 Mention of variable for use as array index 2 Use of a loop (to check through the array) 3 Save index of (first) unused element found 4 If word is the same as the current array element then return `FALSE` / set flag 5 If word not already in array **and** unused element available, store word in unused element and return `TRUE` otherwise return `FALSE` **Note: Max 4 marks**	**4**

Examiner report missing
9618_s21_qp_21 Question 7